Unidad 1
Fundamentos del paradigma orientado a objetos

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



Puesta en común de la clase estudiante

A partir de la actividad práctica que realizaron, vamos a construir juntos la anatomía de nuestra clase Estudiante.

Atributos (Características / Datos)

- legajo
- nombre
- dni
- materias
- carrera
- ...

Comportamientos (Acciones / Métodos)

- mostrarInformacion()
- inscribirseACursada()
- consultarMateriasAprobadas()
- ...

¿Qué datos son indispensables para identificar unívocamente a un estudiante en la universidad?

Si este Estudiante interactuara con un sistema de inscripción, ¿qué acciones necesitaría poder realizar?

Clase

```java
public class Estudiante {
    public String nombreCompleto;
    public int numeroLegajo;

    public void mostrarInformacion() {
        System.out.println("Estudiante: " + nombreCompleto +
        " - Legajo: " + numeroLegajo);
    }
}
```

Esa estructura que crearon, public class Estudiante, es la clase. Es la **plantilla** o el **molde** con el que definimos las características (atributos) y acciones (métodos) de todos los estudiantes.

Objeto

```java
public class SistemaGestionAlumnos {
    Run | Debug
    public static void main(String[] args) {
        Estudiante estudiante1 = new Estudiante();
        estudiante1.nombreCompleto = "Juan Pérez";
        estudiante1.numeroLegajo = 12345;

        Estudiante estudiante2 = new Estudiante();
        estudiante2.nombreCompleto = "Ana Gómez";
        estudiante2.numeroLegajo = 67890;

        estudiante1.mostrarInformacion();
        estudiante2.mostrarInformacion();
    }
}
```

Luego, cuando en main escribieron **new Estudiante()**, crearon un objeto. Esa es la **instancia** concreta, el “estudiante” real que existe en la memoria del programa, con datos concretos.

Atributos (estado)

Los atributos son las características y propiedades que definen a un objeto. Son “**variables internas**” que almacenan la información del objeto. responden a la pregunta: "¿Qué es o qué sabe este objeto?".
La primera instancia anterior era Juan Pérez con legajo 12345.

Métodos (comportamiento)

Los comportamientos son las acciones que un objeto es capaz de realizar. Son las funciones que definen **lo que el objeto sabe hacer**. Por lo general, estos métodos utilizan los atributos internos del objeto para poder funcionar y pueden alterarlos.

Todos los objetos de la clase Estudiante, saben mostrarInformacion().

Paradigma imperativo

Centrado en describir cómo debe realizarse una tarea.

Consiste en una secuencia de instrucciones o comandos que manipulan directamente la información para alcanzar un resultado.

Ventajas:

- **Simple y directo** (prog. chicos)
- **Eficiente**

Desventajas:

- Un estado compartido global crea **dependencias complejas y difíciles de seguir**.
- Un cambio puede tener efectos inesperados en otras partes del programa, **Difícil de Mantener y Escalar.**
- Código muy ligado a la secuencia, **Baja Reutilización.**

Paradigma procedural

Es una evolución del imperativo. El código se organiza en procedimientos (o funciones) que agrupan conjuntos de instrucciones para realizar una tarea específica. **Separación entre los datos y los procedimientos que los manipulan**.

Ventajas:

- **Modularidad:** organizado y legible.
- **Reutilización de Código**: Un mismo procedimiento se puede llamar desde múltiples lugares.
- **Mantenimiento más Sencillo:** Los cambios se pueden aislar dentro de un procedimiento.

Desventajas:

- **Acoplamiento Débil:** Los datos y las operaciones siguen separados. No hay un mecanismo que garantice que solo ciertos procedimientos modifiquen ciertos datos.
- **No Modela el Mundo Real:** El enfoque sigue estando en los procesos, no en las entidades del problema.

Paradigma Orientado a Objetos ¿Qué es?

- Se piensa el sistema como un **ecosistema de objetos autónomos**.
- Cada objeto tiene un rol y unas **responsabilidades**.
- El resultado del programa se logra a través de la **comunicación y colaboración** entre estos objetos.

¿Por qué agrupamos comportamiento y estado?

- Para crear componentes que sean **responsables de sí mismos**.
- Al unir los datos con las operaciones que los manipulan, creamos una unidad coherente y protegida. **Encapsulada**.

Analogía con el mundo real

No hace falta saber cómo las cosas funcionan, solo proveer una interfaz. Auto: acelerar, frenar. El auto se encarga de su complejidad interna.

Paradigma Orientado a Objetos ¿Por qué?

Una respuesta a la complejidad creciente del software

Programa procedural se vuelve difícil de mantener -> POO mejora estructura y gestión de esa complejidad.

Alineación entre el modelo del problema y el modelo de la solución

Se diseñan entidades análogas al mundo real, limitadas por un contexto. Estudiante con estado y que sabe hacer, limitado a un sistema de gestión de la universidad. Esto hace el código más intuitivo y fácil de mantener, incrementado la calidad del software.

Fomenta la Reutilización

la clase Estudiante que crearon es ahora una pieza de software reutilizable en cualquier otra parte del sistema de la universidad que necesite gestionar estudiantes.

Lenguajes de programación

Smalltalk: El pionero y uno de los más "puros". Su filosofía es que "todo es un objeto". Aunque hoy es de nicho, su influencia en los lenguajes e IDEs modernos es inmensa.

Java: Famoso por su portabilidad ("Escribe una vez, ejecuta en cualquier lugar"). Es robusto, seguro y un pilar en el mundo empresarial y Android.

Python: Conocido por su sintaxis simple y legible. Es multiparadigma (soporta POO) y domina en áreas como ciencia de datos, IA y desarrollo web.

C++ / C#: C++ es una evolución de C que añade POO, buscando máximo rendimiento y control. C# es la alternativa de Microsoft a Java, fundamental en el ecosistema .NET y en el desarrollo de videojuegos con Unity.

estudiante1.nombre = "Juan Perez";
¿Qué sucedió? ¿Alguien puede describir el error que mostró el IDE?